

TEST STRATEGY

Car Rental Section — Booking.com (New York)

Document Title:	Test Strategy - Booking.com Car Rental (NY)
Application Under Test:	https://www.booking.com/cars/index.html
Scope Area:	Car rental search & booking flow - New York, USA
Author:	Dmytro Radchenko
Role:	Senior QA Strategist
Version:	1.0
Date:	May 14, 2026
Status:	Approved for execution

1. Scope

This Test Strategy defines the approach, objectives, and quality criteria for verifying the **Car Rental section** of Booking.com (<https://www.booking.com/cars/index.html>) with a focus on the **New York, USA** pickup region. The strategy covers the end-to-end user journey: searching for rental cars, applying filters (supplier, car type, transmission, fuel policy, mileage), comparing offers, reviewing rental terms, completing the booking flow, and managing the reservation.

In scope:

- Search functionality: pickup/drop-off location (New York airports — JFK, LGA, EWR — and downtown locations), date/time pickers, driver age.
- Search results page: sorting, filtering, pagination, map view, supplier comparison, price breakdown.
- Car details page: vehicle specifications, included/excluded items, rental conditions, supplier ratings.
- Booking flow: extras (GPS, child seat, additional driver), insurance offers, driver details, payment, confirmation.
- Account management: My Bookings, cancellation, modification of reservations.
- Cross-functional aspects: localization (EN-US), currency (USD), responsive web design (desktop, tablet, mobile-web), accessibility (WCAG 2.1 AA).
- Non-functional testing: performance, security, compatibility, usability.

Out of scope:

- Native mobile applications (iOS / Android Booking.com app) — covered by a separate strategy.
- Backend supplier integrations beyond contract-level verification (handled by Integration team).
- Loyalty program (Genius) business rules outside of car rental discounts.
- Hotels, Flights, Attractions, and Taxi product lines.

2. References

- ISO/IEC/IEEE 29119-1..5 — Software Testing Standards
- IEEE 829 — Standard for Software and System Test Documentation
- ISTQB Foundation & Advanced Test Manager Syllabi (2018 / 2023)
- WCAG 2.1 Level AA — Web Content Accessibility Guidelines
- OWASP Top 10 (2021) and OWASP ASVS 4.0 — Security baseline
- PCI DSS v4.0 — Payment Card Industry Data Security Standard
- GDPR / CCPA — Privacy regulations applicable to EU and California residents
- Booking.com Internal QA Handbook — **TBD**
- Product Requirements Documents (PRDs) for Cars vertical — **TBD** (Confluence)
- API Specifications: Cars Search API, Booking API, Supplier Inventory API — **TBD** (Swagger/OpenAPI)
- Agile Testing — Lisa Crispin & Janet Gregory (2009 / 2014 editions)

3. Glossary

Term	Definition
AUT	Application Under Test (Booking.com Car Rental web flow)
BDD	Behavior-Driven Development
CI/CD	Continuous Integration / Continuous Deployment
CR	Car Rental
DoD	Definition of Done
DoR	Definition of Ready
E2E	End-to-End test
GDS	Global Distribution System (e.g., Amadeus, Sabre)
NFR	Non-Functional Requirement
PII	Personally Identifiable Information
POC	Proof of Concept
PWA	Progressive Web Application
RBT	Risk-Based Testing
RTM	Requirements Traceability Matrix
SLA	Service Level Agreement
SUT	System Under Test
UAT	User Acceptance Testing
WCAG	Web Content Accessibility Guidelines
XSS / SQLi / CSRF	Cross-Site Scripting / SQL Injection / Cross-Site Request Forgery

4. Objectives

The primary objective is to ensure that the Booking.com Car Rental flow for the New York market is functionally correct, performant, secure, accessible, and delivers an excellent user experience across supported browsers and viewports. Specific objectives are listed below.

- Validate that all user-facing search and booking scenarios in New York work end-to-end without functional defects of Severity 1 or 2.
- Confirm that pricing, currency conversion, taxes, and surcharges are accurate for New York pickup locations (incl. airport fees, NY State sales tax, vehicle license recovery fees).
- Verify supplier (Hertz, Avis, Budget, Enterprise, SIXT, etc.) integrations return correct inventory and that fallbacks are handled gracefully.
- Achieve and maintain at least 85% automated regression coverage of P1/P2 test cases.
- Ensure that page-load performance meets defined SLAs: search results < 3s p95 on 4G; Largest Contentful Paint < 2.5s.
- Validate accessibility against WCAG 2.1 AA — zero critical axe-core violations on key pages.
- Prevent regressions in conversion-critical pages through smoke tests gated in CI/CD pipeline.
- Provide transparent, real-time quality metrics to product, engineering, and business stakeholders.

5. General Strategy

5.1 Test Levels

Testing follows the ISTQB V-model adapted for Agile delivery. Each level has clear ownership and entry/exit criteria:

Level	Owner	Focus	Tooling
Unit	Developers	Pure functions, components, hooks	Jest, Vitest, JUnit
Integration / Component	Developers + QA	Module interaction, API contracts	Pact, REST-assured, Supertest
System / E2E	QA	Full user journey in browser	Playwright, Cypress
Acceptance (UAT)	Product + QA	Business validation, beta cohort	TestRail, BrowserStack live

5.2 Test Types

- **Functional:** search, filters, sorting, booking flow, payment, cancellation.
- **Regression:** automated suite executed nightly and on every PR to *main*.
- **Smoke / Sanity:** 12-15 critical journeys, runs in < 5 minutes on each deploy.
- **Performance:** load and stress tests on Search API and Booking API (k6, JMeter).
- **Security:** SAST (SonarQube), DAST (OWASP ZAP), dependency scanning (Snyk), pen-tests quarterly.
- **Compatibility:** latest 2 versions of Chrome, Firefox, Safari, Edge; iOS Safari 15+; Android Chrome 110+.
- **Accessibility:** axe-core in CI plus manual screen-reader checks (VoiceOver, NVDA).
- **Localization:** EN-US baseline; spot checks for EN-GB, DE, ES (currency, date formats).
- **Exploratory:** time-boxed charter-driven sessions each sprint.
- **Usability:** moderated and unmoderated tests via UserTesting.com — **TBD** frequency.

5.3 Test Environments

Environment	Purpose	Refresh Cadence
DEV	Developer integration, ad-hoc testing	Continuous
QA / STAGING	QA full regression, automation, manual	Per release branch
PRE-PROD	Production-mirror for performance & UAT	Weekly
PROD	Live monitoring, synthetic checks, A/B	N/A

Test data: a curated New York data set including JFK, LaGuardia, Newark airports and 5+ downtown locations, multiple supplier accounts, and pre-staged user accounts (loyalty tiers, payment methods) — managed in a Test Data Management service. PII is masked in non-prod environments per GDPR/CCPA.

6. Risk Management

Risk-Based Testing (RBT) drives prioritization. Risks are evaluated using Likelihood (1-5) and Impact (1-5); the product Risk Score ($L \times I$) determines test depth and automation priority.

#	Risk	L	I	Score	Mitigation
R1	Incorrect pricing for NY pickup (tax/fee miscalculation)	3	5	15	Dedicated pricing test suite; daily reconciliation vs. supplier rates
R2	Supplier API downtime or partial inventory	4	4	16	Contract tests (Pact), circuit-breaker tests, graceful degradation
R3	Payment failure / double charge	2	5	10	Sandbox payment provider tests; idempotency checks; reconciliation
R4	Accessibility violations leading to ADA lawsuits	3	4	12	axe-core gating in CI, quarterly manual audits
R5	Performance regression on Search results page	3	4	12	k6 load tests in pipeline; performance budgets in Lighthouse CI
R6	Cross-browser layout breakage	3	3	9	BrowserStack visual regression (Percy/Applitools)
R7	Data privacy / PII leakage	2	5	10	DAST, masking in lower envs, regular GDPR audits
R8	Time-zone bugs (EST/EDT) on date pickers	3	3	9	Targeted boundary tests around DST transitions

The risk register is reviewed at each sprint planning and updated as features and supplier integrations evolve.

7. Test Selection and Prioritization

Test cases are designed using a combination of equivalence partitioning, boundary value analysis, decision tables, state transition, and pairwise (orthogonal array) techniques. Each test case is assigned a priority level based on business impact and risk exposure:

Priority	Definition	Coverage Target	Execution
P1 — Critical	Conversion-critical paths (search, book, pay, cancel)	100% automated	Every PR & nightly
P2 — High	Filters, sorting, account, extras, insurance	≥ 90% automated	Nightly
P3 — Medium	Cosmetic, secondary flows, localization spot-checks	≥ 60% automated	Weekly / release
P4 — Low	Edge cases, rarely used UI states	Manual or exploratory	Ad-hoc

Selection criteria for each release cycle:

- All P1 tests (smoke + regression) execute on every build (gating).
- P2 tests run nightly and as part of release-candidate validation.
- Impact analysis (code coverage + recent changes) selects relevant P3/P4 tests to add to the next regression cycle.
- Exploratory charters are written per user story to complement scripted tests.
- Tests with a high flakiness rate (> 2%) are quarantined and triaged within 48 hours.

8. Test Documentation and Reporting

Documents produced and maintained throughout the lifecycle:

- **Test Strategy** (this document) — reviewed quarterly.
- **Test Plan** per release — scope, schedule, resources, exit criteria.
- **Test Cases** — authored in TestRail / Xray, traceable to user stories.
- **Requirements Traceability Matrix (RTM)** — linking requirements → tests → defects.
- **Automation Reports** — Allure / Playwright HTML reports stored in CI artifacts.
- **Daily / Sprint Test Summary** — sent to stakeholders via Slack and email.
- **Release Readiness Report** — go/no-go signal at release gate.
- **Defect Reports** — in Jira with full reproduction steps, environment, severity, priority.

Key Quality Metrics:

Metric	Target	Source
Automated regression pass rate	≥ 98%	CI dashboard
Defect leakage to production	< 2% of total defects	Jira
Mean Time to Detect (MTTD)	< 4 hours	Monitoring + CI
Mean Time to Repair (MTTR)	< 24 hours for Sev-1	Jira
Test execution coverage	≥ 95% of planned cases	TestRail / Xray
Flaky test ratio	< 2%	Automation reports
Code coverage (unit + integration)	≥ 80% lines, ≥ 75% branches	SonarQube

9. Test Automation and Tools

Automation follows the test pyramid: a wide base of unit tests, a strong layer of API / contract tests, and a focused set of E2E tests on critical user journeys. The Page Object Model and component-based locators are used to maintain stability across UI refactors.

Category	Tool(s)	Notes
UI E2E Automation	Playwright (TypeScript), Cypress (legacy)	Headless in CI, headed for debugging
API Automation	REST-assured (Java), Postman/Newman, Pact	Schema validation via OpenAPI
Performance	k6, JMeter, Lighthouse CI	Performance budgets gated
Security	OWASP ZAP, SonarQube, Snyk, Trivy	Quarterly external pen-test
Accessibility	axe-core, WAVE, manual screen readers	VoiceOver, NVDA, JAWS
Visual Regression	Applitools / Percy	Critical pages snapshot
Cross-browser / Devices	BrowserStack, Sauce Labs	Real-device cloud
Test Management	TestRail and/or Xray for Jira	Single source of truth
Defect Tracking	Jira	Integrated with TestRail

Category	Tool(s)	Notes
CI/CD	GitHub Actions / Jenkins	Parallelized test shards
Reporting	Allure, Grafana, Slack bots	Live dashboards
Test Data Management	Custom TDM service + Faker	PII-safe data sets

Automation design principles:

- Tests are independent, deterministic, and runnable in any order.
- Test data is generated or seeded via API rather than UI to keep runs fast.
- Locators prefer accessibility attributes (data-testid, role, aria-label).
- Each E2E test must run in < 90 seconds; the full E2E suite must complete in < 25 minutes (parallelized).
- Flaky tests are quarantined automatically; a weekly review session ensures none are forgotten.
- All automation code follows the same review process as production code (Pull Requests + code review).

10. Configuration Management

All test artifacts (test cases, automation code, test data, configuration files, environment definitions) are stored in Git and version-controlled alongside the application source code where possible. The following rules apply:

- Branching follows trunk-based development with short-lived feature branches and mandatory PR reviews.
- Automation code lives in the same monorepo (or sibling repo) and is tagged per release.
- Environment configuration is managed through HashiCorp Vault for secrets and Helm/Terraform for infra.
- Test data sets are versioned and tagged; restoration scripts run on environment refresh.
- Each test execution records: build number, commit SHA, environment, browser version, and test data set hash.
- Baseline images for visual regression are reviewed and approved by QA leads before merge.

11. Incident Management

Defects discovered during testing or in production follow a standard lifecycle: *New* → *Triaged* → *In Progress* → *Ready for Test* → *Verified* → *Closed*. Each defect must include reproduction steps, expected vs. actual result, environment, browser, logs/screenshots/videos, severity, and priority.

Severity	Definition	Response SLA	Resolution SLA
S1 — Critical	Production outage or blocker (cannot book a car)	15 min	4 h
S2 — High	Major function broken with no workaround	1 h	24 h
S3 — Medium	Function broken with workaround, or cosmetic on critical page	1 business day	Next release
S4 — Low	Minor cosmetic / nice-to-have	5 business days	Backlog

Production incidents trigger an Incident Response process led by the on-call engineer, with a blameless post-mortem within 5 business days for S1/S2 events. QA contributes detection and prevention actions to the post-mortem.

12. Sub-process Strategy Details

12.1 Test Planning

- Test plans are created per release / epic, derived from this strategy.
- **Inputs:** PRD, UX designs, API specs, risk assessment.
- **Outputs:** scope, schedule, resource allocation, exit criteria, sign-off.

12.2 Test Design

- Techniques: Equivalence Partitioning, Boundary Value Analysis, Decision Tables, State Transition, Pairwise, Exploratory charters.
- Acceptance criteria written in Gherkin (Given/When/Then) where appropriate.
- Designs reviewed by another QA and one developer (three-amigos refinement).

12.3 Test Implementation & Execution

- Automation written within the same sprint as the feature (Definition of Done includes automation).
- Manual exploratory sessions executed before release-candidate sign-off.
- Test runs orchestrated by CI: smoke on every PR, regression nightly, performance weekly.

12.4 Test Monitoring & Control

- Real-time dashboards on Grafana display pass rate, flaky rate, coverage trend.
- Slack alerts on red builds and on test flakiness spikes.
- Weekly QA sync to review trends and adjust priorities.

12.5 Test Completion

- Release readiness report aggregates results from all test types.
- Outstanding defects above an agreed risk threshold block release.
- Lessons learned captured and fed back into this strategy.

13. Agile Test Strategy Considerations

The Car Rental team operates in 2-week Scrum sprints with continuous delivery to production behind feature flags. The Agile testing approach embraces the following:

- **Whole-team quality ownership:** developers, QA, product, and design share responsibility for quality. QA acts as a quality coach.
- **Shift-Left:** QA participates in story refinement, definition of acceptance criteria, and three-amigos sessions before development starts.
- **Shift-Right:** production observability (synthetic checks, real-user monitoring, log/metric alerts) complements pre-production tests; canary releases and feature flags reduce risk.

- **Test pyramid & quadrants:** Brian Marick's Agile Testing Quadrants guide test type selection; the test pyramid keeps fast feedback at unit/API levels.
- **Definition of Ready (DoR):** story has clear acceptance criteria, test data identified, designs attached, and risks reviewed.
- **Definition of Done (DoD):** code reviewed, unit and integration tests passing, automation added at the appropriate level, accessibility checks green, security scans clean, documentation updated, feature flag default safe.
- **Continuous Testing in CI/CD:** every commit triggers unit, lint, contract, and smoke tests; merge to main triggers full regression, security scans, and Lighthouse audits.
- **Pairing & Mob testing:** QA pairs with developers on automation and with product on exploratory testing.
- **Retrospectives:** quality metrics are a standing item; actions feed back into this strategy.
- **Experimentation:** A/B tests on Car Rental search results are monitored for guardrail metrics (errors, conversion drops) before being rolled out.

13.1 In-sprint test activities

Sprint Day	QA Activity
Day 1-2	Refinement, three-amigos, test idea generation, risk review
Day 3-7	Pair on automation, exploratory testing on emerging functionality
Day 8-9	Regression validation, accessibility/performance smoke, release candidate prep
Day 10	Sprint review demo, retrospective, release sign-off

Document Approval

Role	Name	Signature	Date
Author / Senior QA Strategist	Dmytro Radchenko	_____	May 14, 2026
QA Lead	TBD	_____	TBD
Engineering Manager	TBD	_____	TBD
Product Manager (Cars)	TBD	_____	TBD

Confirm if further adjustments are needed.